

Five-Day Kotlin Ramp-Up Plan

Goal: learn Kotlin properly before a Java/Kotlin eCommerce contract starts, while closing the largest gaps in the job requisition and getting real practice deploying a JVM service to AWS.

Constraints:

- Five days, roughly contiguous.
 - All Kotlin code hand-written. Agents are used for infrastructure only (Days 1, 4, 5), and for *describing* deployment steps rather than performing them on Days 2–3.
 - AWS spend capped at \$100, expected well under \$50.
 - Background: strong Python, rusty Java, existing Terraform / Kubernetes / AWS experience.
-

1. Editor and toolchain

Use IntelliJ IDEA as the primary editor

JetBrains announced the official “Kotlin by JetBrains” VSCode extension at KotlinConf 2026, backed by a Kotlin Language Server built on IntelliJ’s own code-insight infrastructure. It provides completion, diagnostics, navigation, quick-fixes, formatting and project import — but it is still labelled **Alpha**, and JetBrains state plainly that IntelliJ IDEA and Android Studio remain the most complete Kotlin environments.

With only five days, debugging the toolchain is not a good use of any of them. There is also a second reason: in a Kotlin shop the IDE is part of the shared vocabulary. Refactorings, the Java-to-Kotlin converter, coroutine-aware debugging and Gradle sync are things teammates will assume you have.

IntelliJ IDEA Community (free, Kotlin plugin bundled), plus:

- Ktlint or Detekt
- HashiCorp Terraform
- AWS Toolkit
- Docker

VSCode, as a secondary editor

Useful for Terraform, configs and shell work.

Extension	ID / note
Kotlin by JetBrains	<code>jetbrains.kotlin-server</code> — note the marketplace ID changed from the old <code>jetbrains.kotlin</code> ; let it uninstall the old one and reload
Gradle for Java	<code>vscjava.vscode-gradle</code>
Java Extension Pack	needed for the Java debugger to attach to a JVM
Docker, AWS Toolkit, HashiCorp Terraform, EditorConfig	general
Java & Kotlin by IntelliJ IDEA	optional preview; free during preview, requires an IDEA Ultimate licence afterwards

Stack and versions

Spring Boot with Kotlin and coroutines, on Gradle Kotlin DSL. Spring Boot is the highest-probability guess for a JVM backend shop, and everything genuinely Kotlin about the project transfers even if the job turns out to be Ktor or Android. The reverse is not true — Ktor teaches no Spring.

- **Kotlin 2.4** (released June 2026; latest patch 2.4.10)
- **Spring Boot 4.1.1** (August 2026)

- **Java 21 or 25 LTS** — Spring Boot 4.1 supports Java 17 through 26

Wire in on Day 1, because it is what a real repository looks like: Gradle version catalogs (`libs.versions.toml`), `ktlint` or `ktfmt`, `Detekt`, `Kotest` or `JUnit 5`, `MockK`, `Testcontainers`, `Kover`.

2. The project: a catalog personalization service

Frontends request a product catalog together with a customer context. The service returns the catalog filtered and decorated for that customer’s segments, including which CTAs to show.

Three pieces:

1. **A rules engine.** Rules are authored in a Kotlin DSL, compiled to a sealed-class AST, and evaluated against a customer context — e.g. `Segment("vip")` and `(CartValue gte 100.usd) and not Region("EU")`. Rules produce decisions: `Include`, `Exclude`, `ShowCta(id, variant)`, `HideCta(id)`.
2. **A content source.** Product and CTA content lives in Contentful (free tier), fetched through a client you write, with caching and concurrent bulk fetch.
3. **A migration.** A hand-made “legacy Magento-shaped” JSON export — nested, denormalized, inconsistently typed, nullable everywhere — transformed into the Contentful content model. You do not need to run Magento. You need to have made the data-modeling decisions and be able to defend them.

One addition, given that Java is rusty: write the legacy Magento adapter in **Java**, calling into the Kotlin domain. It is small and contained, it knocks the rust off, and Kotlin/Java interop at a real seam — platform types, `@Nullable` annotations, `@JvmStatic` — is something you will hit in week one on a mixed codebase.

How it maps to the requisition

Requisition item	Your note	What the project gives you
Catalog filtering by segmentation	OK	The built artifact, not just the claim
Rules engine for CTA show/hide	OK	Same — this is the core of the project
Third-party integration at scale	No large-scale experience	A real client with caching, timeouts, bulk fetch; measured numbers, not scale claims
Magento → Contentful migration	None	Hands-on Contentful modeling and a written transform
Segmentation / personalization patterns	None	Closed
CTA / experimentation frameworks	None	Partially closed via CTA decisions and variants

Why it is also the better Kotlin exercise

- **Sealed AST plus exhaustive when** is the spine of the project, not a garnish. Add a rule type and the compiler walks you through every evaluator, serializer and explainer that needs updating.
- **Type-safe builders** with receivers, `@DslMarker` and infix functions — Kotlin doing something Python structurally cannot.
- **Null safety at a hostile boundary.** Contentful payloads are deeply optional by design; the Magento export is worse. Parsing that into non-null domain types is the exercise that makes the type system click.
- **Coroutines** for concurrent content fetch, cache warming and timeouts.
- **Value classes** for `SegmentId`, `Sku`, `CtaId`.

3. The five days

Hand-writing all the Kotlin cuts throughput to roughly a third of agent-assisted pace. That is the right trade for learning and the wrong one for scope, so this plan is deliberately small and the rules engine gets the protected time.

Day 1 — agent-assisted, infrastructure only

Gradle Kotlin DSL project, version catalog, ktlint/Detekt/Kover, Spring Boot skeleton, one endpoint, containerized, deployed to EC2 and reachable.

Also: create the Contentful space and define the content model **by hand in their web UI**, then enter a dozen products. This is not busywork — it is the thing you have no experience with.

Deploying something trivial on Day 1 is what makes “iterating on deployments” possible by Day 3. Deploying on Day 4 means one deployment, not iteration.

Day 2 — hand-written: domain and rules AST

Sealed hierarchy for rule expressions and decisions, the evaluator, exhaustive `when` everywhere, `kotlinx.serialization` for the JSON form of a ruleset. Tests written by hand as you go, because writing Kotlin test code is how you learn the assertion and property-based idioms.

Deliberate exercise: write the evaluator once the way you would in Python (dicts and string tags), then again with sealed classes, then delete the first. The comparison is the lesson.

Day 3 — hand-written: the DSL and the CMS client

The type-safe builder for authoring rulesets, then the Contentful client with coroutine-based bulk fetch and a cache.

Deployment work today follows your rule: ask an agent for a deployment script, read it, then run it yourself. Redeploy at least twice.

Day 4 — mixed: migration and integration

The Java adapter reading the fake Magento export, mapping into the content model, writing into Contentful. This is where the null-handling and interop lessons land.

Agent help is fine for Contentful Management API mechanics. Write the mapping logic by hand — that is where the modeling judgment lives.

Day 5 — agent-assisted: deployment iteration and the scaling story

CI on push, health and readiness endpoints, structured logging, Micrometer. Then run a load test (k6 or Gatling) against the deployed service, with and without the cache, and write down the p50 and p99 numbers.

This targets the one genuine gap in the requisition. You cannot acquire large-scale experience in five days and should not pretend to. What you *can* have by Friday is a measured before-and-after on one caching decision, plus the vocabulary that goes with it: cache-aside versus write-through, bulk fetch to avoid $N+1$ against the CMS, circuit breaker and bulkhead around a third-party dependency, and what evaluation cost does as ruleset size grows. “I measured it, and here is what happened” is a far better answer in a design conversation than a scale claim you cannot back.

4. Scope discipline

Decide the cut list now, not at 11pm on Thursday.

Protect: the sealed AST, the DSL, the evaluator, and at least three deployments.

Cut freely if behind:

- Postgres — use in-memory state plus Contentful as the source of truth; persistence teaches you nothing new here
 - Authentication
 - Any admin UI
 - More than three rule operators
 - The Java adapter
 - The GraalVM native-image experiment — fun, but not in this requisition
-

5. Cost and guardrails

Roughly \$2–\$5 for the week if you drop Postgres, and comfortably under \$50 even if you do not.

- Use `t4g.small`, not `t3.micro`. A JVM on 1 GB is miserable and Gradle builds on a micro instance are worse. Build locally or in GitHub Actions and ship artifacts; do not compile on the box. Roughly \$12/month on-demand, so about \$2 for five days.
- **Skip EKS.** You already know it, and the control plane alone is about \$72/month — it would eat the entire budget for capability you do not need here.
- **Avoid NAT Gateways.** A public subnet with a security group is fine for a personal project. A NAT Gateway is about \$32/month plus data charges, and is the most common way a budget like this evaporates unnoticed.
- **Postgres, if kept:** either a container on the same instance (cheapest) or `db.t4g.micro` RDS at around \$12/month if you would rather practise the managed-service wiring.
- **Set an AWS Budgets alert at \$25 on Day 1**, and write the teardown (`terraform destroy`) before you need it.

Contentful: the free tier costs nothing but has real ceilings on content types, records and monthly API calls, and the published limits have moved around over the past couple of years — check current numbers when you sign up. Watch the Day 5 load test: point it at cached responses rather than straight through to Contentful, or you will burn the month's API quota in about four minutes. That is also a legitimately good reason to build the cache before the load test rather than after.

6. One habit worth keeping

The temptation with a language you already half-know from Java is to write Java-with-different-syntax and call it done. The fastest correction is a running list of “I reached for X — what is the Kotlin way?” moments, with twenty minutes each evening spent looking them up.

Coming from Python, the biggest adjustments are usually exhaustiveness checking, immutability by default (`val`, `List` versus `MutableList`), and the expectation that the type system encodes things you would otherwise handle with a docstring and a prayer.